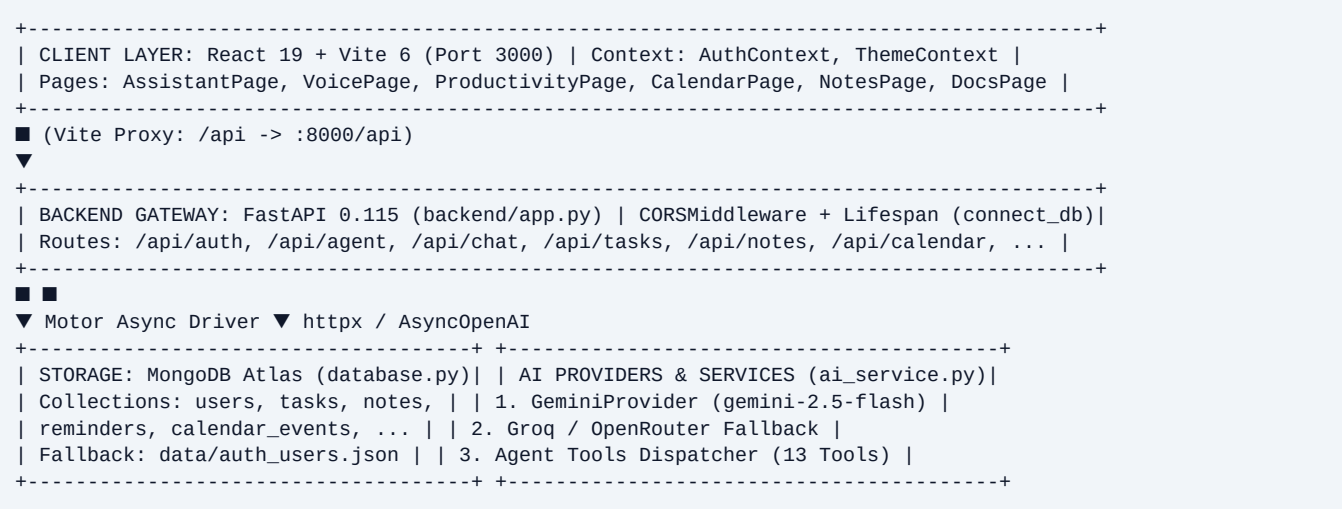
ASTRA AI OPERATING SYSTEM

Jarvis AI Productivity Assistant — Comprehensive Technical Architecture & Codebase Audit

Target Codebase: Full Project (Frontend + Backend)	**Backend Stack:** FastAPI 0.115 + Motor (MongoDB) + Async Python
Frontend Stack: React 19 + Vite 6 + Tailwind CSS	**AI Infrastructure:** Google Gemini 2.5 + Groq + OpenRouter + Multi-Tool Router
Voice Engine: Web Speech STT/TTS + Async Voice Service	**Audited Modules:** 25+ Files (Models, Providers, Services, Routes, Views)

1. System Architecture & High-Level Data Flow

Astra AI OS is structured as a decoupled client-server distributed application. The frontend is a single-page application built on React 19, communicating via HTTP REST (with JWT bearer authentication) and Web Speech APIs to an asynchronous Python FastAPI microservice backend. The backend orchestrates database interactions via Motor (async MongoDB driver) and delegates LLM reasoning through a sequential AI Provider Router.



2. Frontend Technologies, Routing & State Management

Core Technologies: React 19.0.0, Vite 6.0.0, React Router DOM 6.28.0, Tailwind CSS 3.4.17, Lucide React, and Firebase 12.18.0.

Layout Architecture (src/App.jsx): Divided into public promotional pages (/, /features, /pricing, /docs, /login, /signup) and the authenticated application shell (/app) wrapped in AppLayout (Sidebar navigation, TopHeader, and content Outlet).

State Management (React Context):

- **AuthContext (src/context/AuthContext.jsx):** Persists current user session. On mount, validates JWT token via apiGetMe(). Attaches Firebase onAuthStateChange listener to synchronize Google OAuth popups with the backend. Provides auto-registration fallback on login 401.
- **ThemeContext (src/context/ThemeContext.jsx):** Persists Light, Dark, or System mode to localStorage, binding to window.matchMedia('(prefers-color-scheme: dark)') and injecting custom accent color CSS variables.
- **ToastContext (src/context/ToastContext.jsx):** Global notification queue supporting Success, Error, and Info toast alerts with automatic 4-second expiration.

3. Backend Technologies, Structure & Services

Core Technologies: FastAPI 0.115+, Uvicorn 0.30+, Motor 3.5+ (MongoDB async), Pydantic 2.8+, python-jose (JWT), bcrypt 4.0+, httpx 0.27+.

Application Architecture:

• backend/app.py: Initializes FastAPI application with async contextmanager lifespan for database connections and indexes.
• backend/config.py: Centralized Pydantic BaseSettings singleton with LRU caching for environment secrets.
• backend/database.py: Motor connection pooling, Atlas cluster startup marker (_jarvis_metadata), and index setup.
• backend/services/ai_service.py: Central AI agent orchestrator executing multi-turn tool calling (up to 6 loops).
• backend/services/agent_tools.py: 13 OpenAI-format JSON schemas and corresponding async database handlers.
• backend/services/weather_service.py: Async client for OpenWeatherMap with automatic US city name suffix sanitization.
• backend/services/news_service.py: Async client for NewsAPI top headlines filtering.
• backend/services/voice_service.py: Async dispatcher bridging transcribed audio text to AI agent execution.

4. Database Collections, Schemas & CRUD Operations

MongoDB Atlas is the primary database. On application startup, indexes are automatically verified and created for optimal performance.

Collection	Model / File	Indexed Fields	Key Attributes
users	UserCreate / user.py	email (unique)	name, email, hashed_password, provider, created_at
profiles	ProfileUpdate / profile.py	user_id (unique)	user_id, name, email, avatar_url, bio, phone, location
tasks	TaskCreate, TaskUpdate / task.py	user_id, status, due_date	title, description, due_date, priority (low/med/high), status (todo/in_progress/done)
notes	NoteCreate, NoteUpdate / note.py	user_id, created_at	title, content, tags (array of strings), timestamps
reminders	ReminderCreate / reminder.py	user_id, due_time	message, due_time, repeat (daily/weekly), is_active
calendar_events	EventCreate / routes/calendar.py	user_id, start_time	title, description, start_time, end_time, color (hex)
chat_messages	ChatMessage / routes/chat.py	user_id, created_at	user_id, role (user/assistant), content, created_at
settings	Inlined / routes/auth.py	user_id (unique)	theme, language, notifications_enabled, default_city

5. Authentication & Authorization Architecture

• **Password Security:** Plaintext passwords are salted and hashed with direct bcrypt (bcrypt.hashpw) using 12 rounds.
• **JWT Issuance (backend/utils/auth.py):** Successful authentication signs an HS256 JWT access token with a 24-hour expiration containing sub (user ID) and email.
• **Google OAuth Exchange:** In routes/auth.py:google_auth(), the client passes Google credentials. The backend verifies/upserts the account in MongoDB and returns a standard JWT.
• **Guest User Fallback:** In utils/auth.py:get_current_user(), if an Authorization token is missing or invalid, it returns DEFAULT_GUEST_USER rather than throwing an unhandled 401 error.

6. Pluggable AI Provider Architecture & Tool Execution

The AI subsystem utilizes an abstract Provider Chain (backend/providers/base.py). A central ProviderRouter (backend/providers/router.py) sequentially evaluates providers: **Gemini (Primary) -> Groq (Fallback 1) -> OpenRouter (Fallback 2) -> OpenAI (Optional)**.

• **Error Handling:** Transient failures (429 Rate Limit, 503 Service Unavailable) trigger automatic fallback to the next provider with identical conversation history and tool schemas.

• **Multi-Tool Loop (ai_service.py):** When an LLM requests function execution, the loop runs up to 6 iterations, invoking tools in agent_tools.py, storing results with role: 'tool', and re-querying the model for a synthesized natural language response.

• **Available Tools:** create_task, view_tasks, update_task, delete_task, create_note, view_notes, update_note, delete_note, create_reminder, view_reminders, update_reminder, delete_reminder, view_calendar_events, create_calendar_event, get_weather, get_news, summarize_document.

7. Voice Assistant Architecture & Complete Flow

1. **Client Capture (VoicePage.jsx):** Browser SpeechRecognition (Web Speech API) captures user speech in English without requiring server-side Whisper or audio streaming.
2. **Agent Processing:** The final transcribed text is sent to POST /api/agent/chat.
3. **Tool Execution:** AIService executes any requested tools (e.g., creating tasks, checking weather) and returns a formatted text reply.
4. **Speech Synthesis:** VoicePage.jsx:speakText() initializes a SpeechSynthesisUtterance, speaking the response while synchronizing an animated 60fps pulsing orb and audio waveform bars.
5. **Audio Controls:** The user interface provides explicit *Listen*, *Stop*, and *Retry Audio* buttons.

8. Complete API Endpoint Catalog

Method	Route Path	Auth	Source Handler	Description
POST	/api/auth/register	No	routes/auth.py	Register new account with bcrypt password
POST	/api/auth/login	No	routes/auth.py	Authenticate and return JWT access token
POST	/api/auth/google	No	routes/auth.py	Verify & upsert Google OAuth user record
GET	/api/auth/me	Yes	routes/auth.py	Return current authenticated user profile
POST	/api/agent/chat	Optional	routes/agent.py	Autonomous multi-tool AI Agent endpoint
POST	/api/chat	Optional	routes/chat.py	Standard conversational chat turn
GET	/api/chat/history	Optional	routes/chat.py	Retrieve past conversation turns (default 50)
DELETE	/api/chat/history	Optional	routes/chat.py	Purge chat history for user
GET	/api/tasks	Optional	routes/tasks.py	List tasks, optional ?status= filter
POST	/api/tasks	Optional	routes/tasks.py	Create a task with priority and due date
PUT	/api/tasks/{id}	Optional	routes/tasks.py	Update task title, status, or priority
DELETE	/api/tasks/{id}	Optional	routes/tasks.py	Delete task document by MongoDB ID
GET	/api/notes	Optional	routes/notes.py	List user notes ordered by creation date
POST	/api/notes	Optional	routes/notes.py	Create note with tags array
PUT	/api/notes/{id}	Optional	routes/notes.py	Update note title, content, or tags
DELETE	/api/notes/{id}	Optional	routes/notes.py	Delete note document by ID
GET	/api/calendar/events	Optional	routes/calendar.py	List calendar events with start/end range
POST	/api/calendar/events	Optional	routes/calendar.py	Create calendar event with color code
DELETE	/api/calendar/events/{id}	Optional	routes/calendar.py	Delete calendar event by ID
GET	/api/reminders	Optional	routes/reminders.py	List reminders sorted by due time
POST	/api/reminders	Optional	routes/reminders.py	Create reminder with repeat pattern
DELETE	/api/reminders/{id}	Optional	routes/reminders.py	Delete reminder by ID
GET	/api/weather	No	routes/weather.py	Current weather & 5-day daily forecast
GET	/api/news	No	routes/news.py	Top news headlines by category
POST	/api/summarize	No	routes/summarize.py	AI structured document summarizer

9. Feature Implementation Status Matrix

Feature Module	Implementation Status	Technical Audit Details
AI Assistant & Tools	**FULL**	Autonomous multi-turn agent with 13 tools, context memory, and model picker.
Voice Assistant	**FULL**	Web Speech STT/TTS with pulsing orb, waveform bars, and replay button.
Tasks / Productivity	**FULL**	CRUD operations, priority filtering, and BentoGrid inline integration.
Calendar & Notes	**FULL**	Full CRUD, search, date filtering, modal editors, and color tagging.
Weather & News	**FULL**	Async httpx services with geolocation reverse-coding and fallback mocks.
Doc Summarizer	**FULL**	FileReader text extraction + OpenAI structured JSON or word-count fallback.
Focus & Deep Dive	**MOCK / PARTIAL**	UI tab toggle in TopHeader.jsx only; does not affect app views or task filters.
Settings AI Providers	**MOCK / PARTIAL**	Toggles stored in local React state only; does not update backend .env or DB.
Reminders UI	**PARTIAL**	Backend routes and AI tools exist, but no dedicated page in frontend Sidebar.

10. Top Technical Issues, Vulnerabilities & Recommendations

1. Task Status Enum Incompatibility (HTTP 422 Bug):

- *Location:* frontend/src/pages/ProductivityPage.jsx:51 and BentoGrid.jsx:68 vs backend/models/task.py:12-16.
- *Issue:* The frontend sends status: 'completed' when toggling a task. However, Pydantic's TaskStatus enum only allows 'todo', 'in_progress', and 'done'. This causes FastAPI to reject task completions with HTTP **422 Unprocessable Entity**.
- *Fix:* Change frontend update payloads to send 'done' instead of 'completed'.

2. Document Summarizer Ignores Provider Router:

- *Location:* backend/routes/summarize.py:52-54.
- *Issue:* Directly instantiates AsyncOpenAI(api_key=settings.OPENAI_API_KEY) instead of using the pluggable AIService or ProviderRouter. Users configured with GEMINI_API_KEY will fail to receive AI summaries and get the static word-count fallback.
- *Fix:* Route document summarization requests through ai_service.generate() with structured JSON instructions.

3. Permissive Guest User Fallback Masks Authentication Failures:

- *Location:* backend/utils/auth.py:69-82.
- *Issue:* When an expired or invalid JWT is supplied, get_current_user catches JWTError and silently returns DEFAULT_GUEST_USER. This allows expired sessions to write data under guest_user_default_id, leading to shared state and data pollution.
- *Fix:* Differentiate between unauthenticated optional endpoints and strictly protected mutation endpoints (which must throw HTTP 401).

4. Test Assertion Mismatch in Provider Router Test:

- *Location:* backend/tests/test_provider_router.py:89-97 vs backend/services/ai_service.py:45-49.
- *Issue:* test_ai_service_returns_configuration_message_without_fallback expects 'Gemini is not configured.', but ai_service.py catches all exceptions and unconditionally returns 'AI services are temporarily unavailable. Please try again later.'.
- *Fix:* Update ai_service.py to preserve and surface specific AIProviderConfigurationError messages.

5. Legacy Desktop Voice Assistant Variables in .env.example:

- *Location:* backend/.env.example:27-48.
- *Issue:* Variables like PICOVOICE_ACCESS_KEY, EMAIL_ADDRESS, SMTP_SERVER, WAKE_WORD, and TTS_RATE belong to an older desktop assistant and are not used by the current Web Speech API implementation.
- *Fix:* Clean up .env.example to reflect only variables actively referenced by config.py.

11. Scalability, Performance & Deployment Architecture

- **Production Serving:** Run Uvicorn workers behind Gunicorn (e.g. gunicorn backend.main:app -w 4 -k uvicorn.workers.UvicornWorker) in Docker containerized deployments (AWS ECS, GCP Cloud Run).
- **Frontend Delivery:** Build static assets via npm run build into frontend/dist and serve over global CDN edge networks (Cloudflare Pages, Vercel, AWS S3 + CloudFront).
- **Database Scaling:** Utilize a dedicated MongoDB Atlas cluster (M10+) with connection pooling and replicas.
- **Chat Memory Optimization:** For enterprise workloads, offload recent message context querying from MongoDB to an in-memory Redis cluster.